

MirrorFuzz: Leveraging LLM and Shared Bugs for Deep Learning Framework APIs Fuzzing

Shiwen Ou, Yuwei Li, Lu Yu, Chengkun Wei, Tingke Wen, Qiangpu Chen,
Yu Chen, Haizhi Tang, and Zulie Pan

Abstract—Deep learning (DL) frameworks serve as the backbone for a wide range of artificial intelligence applications. However, bugs within DL frameworks can cascade into critical issues in higher-level applications, jeopardizing reliability and security. While numerous techniques have been proposed to detect bugs in DL frameworks, research exploring common API patterns across frameworks and the potential risks they entail remains limited. Notably, many DL frameworks expose similar APIs with overlapping input parameters and functionalities, rendering them vulnerable to shared bugs, where a flaw in one API may extend to analogous APIs in other frameworks. To address this challenge, we propose MirrorFuzz, an automated API fuzzing solution to discover shared bugs in DL frameworks. MirrorFuzz operates in three stages: First, MirrorFuzz collects historical bug data for each API within a DL framework to identify potentially buggy APIs. Second, it matches each buggy API in a specific framework with similar APIs within and across other DL frameworks. Third, it employs large language models (LLMs) to synthesize code for the API under test, leveraging the historical bug data of similar APIs to trigger analogous bugs across APIs. We implement MirrorFuzz and evaluate it on four popular DL frameworks (TensorFlow, PyTorch, OneFlow, and Jittor). Extensive evaluation demonstrates that MirrorFuzz improves code coverage by 39.92% and 98.20% compared to state-of-the-art methods on TensorFlow and PyTorch, respectively. Moreover, MirrorFuzz discovers 315 bugs, 262 of which are newly found, and 80 bugs are fixed, with 52 of these bugs assigned CNVD IDs.

Index Terms—Deep learning frameworks, shared bugs, bug detection, fuzzing.

I. INTRODUCTION

DEEP learning techniques significantly impact real-world scenarios, including healthcare [1], [2], education [3], [4], finance [5], and autonomous driving [6]. Recent studies [7]–[12] highlight that DL frameworks (e.g., PyTorch [13] and TensorFlow [14]), which serve as the foundational infrastructure for various DL applications, are vulnerable to security flaws. These flaws can result in unexpected consequences in higher-layer applications, such as performance degradation [9], [15], incorrect functionality [16], [17], and memory errors [8].

Shiwen Ou, Yuwei Li, Lu Yu, Tingke Wen, Qiangpu Chen, Yu Chen, Haizhi Tang, and Zulie Pan are with the College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China (e-mail: {oushiwen, liyuwei, yulu, wentk, cy, panzulie17}@nudt.edu.cn; {chenqiangpu, tang-haizhi}@foxmail.com).

Chengkun Wei is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: weichengkun@zju.edu.cn).

A. Shared Bugs: Threats from Commonalities between DL Frameworks

There is a surge of research on testing DL frameworks [18], which predominantly focuses on two categories: model-level and API-level fuzzing. Model-level fuzzing [19]–[23] builds DL models as test cases and uses differential testing to identify defects, but it can only cover a limited set of APIs. In contrast, API-level fuzzing [8], [9], [24]–[32] generates valid API invocation sequences, enabling broader API coverage without the need to construct full DL models, thus attracting attention. Recent advancements in Large Language Models (LLMs) have also introduced LLM-based techniques to enhance fuzz testing of DL frameworks and beyond [7], [30], [31], [33]–[35]. However, while these techniques have proven to be effective, they overlook an important factor: the commonalities between different DL frameworks. The security risks associated with similar APIs within and across frameworks remain under-explored.

Observations: Shared Bugs Within or Across DL frameworks. It is worth noting that vulnerabilities in one software system can often be identified by referencing known issues in other systems that share or reuse source code, libraries, or specifications [36], [37]. With the hypothesis that sharing software vulnerabilities, referred to as **shared bugs**, we conducted an empirical study on several DL frameworks, including PyTorch and TensorFlow. We observe that DL frameworks exhibit similarities in features, architecture, and design, with comparable APIs prone to analogous bugs. The shared bugs can enable early detection within or across DL framework APIs. For instance, as shown in Figure 1, PyTorch's `torch.nn.functional.avg_pool2d` and TensorFlow's `tf.nn.avg_pool3d` APIs for pooling operations are susceptible to the same issue. Setting the `stride` parameter in `avg_pool2d` to 0, an invalid value, causes a crash in PyTorch, which can similarly be reproduced in TensorFlow's `avg_pool3d` API under equivalent conditions.

However, existing DL framework fuzzing approaches overlook the similarities between DL frameworks and the potential security risks these similarities may introduce and fail to leverage shared bugs for early bug detection.

Challenges. In this work, we aim to fuzz the APIs of DL frameworks by leveraging shared bugs. To achieve this, we have to solve the following challenges.

C1: How to recognize the buggy API in the bug report? The reported issues repository (e.g., GitHub) contains extensive

```

1 import torch # PyTorch version 1.5.0
2 import numpy as np
3 input = torch.tensor(np.random.rand(10, 3, 20, 20))
4 kernel_size = 4
5 torch.nn.functional.avg_pool2d(input, kernel_size, stride=0)

-----
1 import tensorflow as tf # TensorFlow version 2.1.0
2 import numpy as np, similar api
3 input = tf.constant([[[[10]]]]], dtype=np.float64)
4 tf.nn.avg_pool3d(input=input, ksize=1, strides=0,
padding='SAME')

```

cause of crash

share the same bug

Fig. 1. Examples of shared bugs.

bug reports related to framework APIs. However, recognizing buggy APIs is challenging due to deviations from community standards, ambiguous descriptions, incomplete code, and inconsistent report formats across different DL frameworks.

C2: How to match similar APIs within and across frameworks? Due to the diversity of APIs (such as differences in documentation, naming conventions, parameters, and return types), it is difficult to match similar APIs within and across frameworks. Even when two APIs are functionally equivalent, their details may still differ across frameworks.

C3: How to leverage shared bugs to synthesize test case that triggers similar bugs? Bugs found in the APIs of one framework may also manifest in similar APIs of other frameworks. Nevertheless, crafting test cases (i.e., code with API calls) that adhere to the target framework's API constraints while effectively reproducing such bugs remains a significant challenge.

B. MirrorFuzz: A Fuzzer that Leveraging Shared Bugs Across DL Framework APIs

Our Approach. To address these challenges, we introduce MirrorFuzz, an automated fuzzing approach powered by LLM to detect bugs in DL frameworks. We propose and implement two techniques for effectively uncovering shared bugs in DL framework APIs. First, we leverage the strengths of LLM in fuzzing DL framework APIs. Specifically, we use LLM for semantic recognition of bug reports, extracting buggy APIs from reported issues. In addition, the ability of the LLM to generate structured high-quality outputs that comply with the target's input constraints enables the creation of precise test cases. Second, we exploit the characteristics of shared bugs to identify the threat that an API may be susceptible to bugs in similar APIs within and across frameworks. This approach is based on the intuition that an input causing a crash in one software is likely to trigger similar failures in other software exhibiting comparable behavior or design patterns. This allows us to efficiently determine if the same bug exists across multiple DL frameworks, saving significant time by reducing the need for extensive fuzzing.

We design MirrorFuzz with three modules: *Buggy APIs Recognizer*, *Similar APIs Matcher*, and *Test case Synthesizer*. The *Recognizer* collects bug reports from GitHub and employs LLM to identify buggy APIs (see Section III-A). Next, the *Matcher* identifies similar APIs across frameworks based on text and semantic similarity (see Section III-B). Notably, we classify the matched similar APIs into *parameter-similar* APIs

and *operation-similar* APIs, according to shared bug manifestations (see Section II-B). Finally, the *Synthesizer* generates test cases to detect bugs by utilizing the outputs from the previous modules (see Section III-C).

Evaluation. We implement a prototype of MirrorFuzz and evaluate it on four popular DL frameworks, i.e., TensorFlow [14], PyTorch [13], OneFlow [38], and Jittor [39]. Although these frameworks are extensively tested in prior studies, MirrorFuzz still successfully discovered 315 bugs in these DL frameworks, of which 262 were previously unknown. We responsibly report these bugs to the corresponding developers in time. As a result, 52 bugs are assigned CNVD IDs, 180 bugs are officially confirmed, and 80 bugs are fixed by the developers. We compare MirrorFuzz with the state-of-the-art DL fuzzing methods, including TENSORSCOPE [9], TitanFuzz [30], Orion [32] and MoCo [40]. MirrorFuzz improves code coverage by 39.92% on TensorFlow and 98.20% on PyTorch, compared to the best-performing baseline. Under the same time budget, MirrorFuzz tests more APIs and detects more unique bugs than all baselines, demonstrating its stronger efficiency and bug-finding capability. More importantly, our research shows that bugs in leading frameworks like TensorFlow and PyTorch can spread to many other frameworks. Also, bugs in smaller or newer frameworks like OneFlow and Jittor may affect mainstream ones, revealing the broad impact of shared bugs across DL frameworks. In summary, we make the following contributions:

- 1) We reveal shared bugs within and across DL frameworks. The high functional similarity across frameworks leads to many similar APIs. More importantly, APIs within a framework are also vulnerable to bugs found in other similar APIs, i.e., similar bugs may be shared across frameworks.
- 2) We implement MirrorFuzz, an automated fuzzing solution that leverages shared bugs and LLM to detect flaws in similar APIs within and across frameworks. Using LLMs' understanding of DL frameworks, we identify buggy APIs from bug reports and generate test cases to expose analogous bugs in target APIs. We open-source the code of MirrorFuzz, the bug list, and data in [41].
- 3) We evaluate MirrorFuzz on four popular DL frameworks and discover a total of 315 bugs. Among these, 262 are newly discovered, and 180 are officially confirmed. We also obtain 52 CNVD IDs.

II. BACKGROUND & PROBLEM STATEMENT

A. Fuzzing DL Framework

Basics about DL Framework. DL frameworks are designed to construct various DL models and applications. Specifically, these frameworks provide essential building blocks for designing, training, and validating DL models through a wide variety of APIs. Thus, developers create DL programs using the provided APIs, which can then be executed with training data to build models and develop applications [17]. With features such as automatic differentiation, GPU/TPU hardware acceleration, and efficient data manipulation, DL frameworks

```

1 # A bug in the torch.nn.AdaptiveMaxPool2d API.
2 import torch
3 size = 2 ** 32
4 m = torch.nn.AdaptiveMaxPool2d(output_size=size,
5                               return_indices=False)
6 inputs = torch.randn(1, 64, 8, 9)
7 m(inputs) # bug
8
9 # A bug in the AdaptiveAvgPool1d API.
10 import oneflow as flow
11 size = 2 ** 32
12 m = flow.nn.AdaptiveAvgPool1d(output_size=size)
13 inputs = flow.Tensor(flow.randn(1, 64, 8))
14 m(inputs) # bug

```

Listing 1. Example of shared bugs in operation-similar APIs across DL frameworks.

```

1 # A bug in the torch.fft.irfftn API.
2 import torch
3 input = torch.randn(10, 512, 512)
4 output = torch.fft.irfftn(input) # bug
5
6 # A bug in the torch.fft.hfftn API.
7 import torch
8 input = torch.randn(10, 512, 512)
9 output = torch.fft.hfftn(input) # bug

```

Listing 2. Example of shared bugs in operation-similar APIs within the DL framework.

significantly enhance the efficiency and flexibility of developing and deploying DL applications. With the growing diversity of requirements, numerous open-source DL frameworks [42]–[44] have emerged. These frameworks commonly provide core functionalities such as tensor computation, automatic differentiation, and model training.

Model-level and API-level Fuzzing. Bug detection in DL frameworks mainly employs fuzzing techniques, which are divided into model-level and API-level approaches. The model-level fuzzing approach primarily involves generating arbitrary complete DL models and feeding them into different backends, such as various frameworks (e.g., TensorFlow and PyTorch) or computing devices (e.g., CPU/GPU), using differential testing to detect crashes, inconsistencies, or performance issues within the frameworks. However, its reliance on DL models as input in differential testing limits API coverage and is affected by precision loss [25]. In contrast, the API-level approach focuses on a specific API and generates complex API sequences to discover bugs within the framework by satisfying the constraints inferred from the API. The API-level fuzzing approach removes the need to build complete DL models, enabling broader coverage of framework APIs, and has thus garnered widespread attention [18].

LLM for Fuzzing. As LLMs continue to evolve, researchers are using them for fuzz testing DL frameworks [7], [30], [31], [33], for tasks such as seed generation, test case mutation, and constraint extraction. LLMs trained on vast amounts of text data have demonstrated powerful performance in natural language processing tasks. Additionally, some variants of LLMs (such as CodeLlama [45], CodeGemma [46], and Qwen2.5-Coder [47]) are also trained and fine-tuned on specific code snippet datasets, enabling them to handle code-related tasks (such as program analysis, code generation, etc.). This allows

LLMs to generate code snippets that satisfy the complex syntax and semantic requirements of DL framework APIs [30].

B. Problem Statement

New Threat. DL framework APIs enable users to interact with the framework for tasks like dataset processing, model building, training, evaluation, and inference. Interestingly, APIs from different frameworks are often developed independently but share similar functionality and design. We observe significant commonalities across different DL framework APIs, driven by three main factors. First, many DL frameworks share essential operations such as tensor computation, automatic differentiation, and dataset handling. Second, the underlying logic of supported algorithms is consistent, such as the mathematical principles behind convolution and pooling in convolutional neural networks (CNNs [48]). Finally, the enhancement of interoperability between frameworks has driven API consistency. For example, most frameworks now support the ONNX standard [49] to facilitate model import, export, and conversion. Due to code reuse and compatibility, there are also many similar APIs within the framework itself. Inspired by the well-known hypothesis that similar systems may exhibit similar bugs, we speculate that APIs within and across DL frameworks may share bugs, and we investigate this through empirical study.

Motivation Example. We collect API bug reports from GitHub Issues for four popular DL frameworks, including TensorFlow [14], PyTorch [13], OneFlow [38], and Jittor [39]. We observe that a shared bug refers to a defect occurring in similar APIs within or across frameworks. These defects are often caused by common designs, logic, or implementation patterns, resulting in similar issues. Moreover, we analyze the occurrence of these shared bugs as follows.

Operation Similarity APIs. We find that APIs with similar operations often share similar bugs, both within and across frameworks. For example, as shown in Listing 1, `torch.nn.AdaptiveMaxPool2d` and `flow.nn.AdaptiveAvgPool1d` triggered the same crash due to an excessively large `output_size` parameter. While one performs max pooling and the other average pooling, both are pooling operations for downsampling feature maps. Therefore, we categorize this type of API as *operation-similar* APIs. A pair of APIs is classified as *operation-similar* if they perform similar functions and produce comparable outputs when provided with analogous inputs. Similarly, Listing 2 shows that *operation-similar* APIs within the same framework can also share bugs.

Parameter Similarity APIs. We observe that APIs within or across frameworks may share certain parameters, even though their functionality or operations differ significantly. Interestingly, these shared parameters often result in similar bugs. For example, both `torch.nn.Conv2d` and `torch.nn.functional.avg_pool2d` crash when the `stride` is set to zero. Despite performing different operations (padding and pooling), the crash is triggered by the same parameter with the same underlying cause. In PyTorch version 1.2, the bug in `torch.nn.Conv2d` gets fixed, and

```

1 # A bug in the torch.nn.Conv2d API.
2 import torch
3 from torch import nn
4 input = torch.randn(1, 1, 32, 32)
5 c = nn.Conv2d(in_channels=1, out_channels=32,
6               kernel_size=4, stride=0, bias=False, padding=(0,
7               1), padding_mode='constant')
8 c(input) # bug
9
10 # A bug in the avg_pool2d API.
11 import torch
12 input = torch.randn(10, 3, 20, 20)
13 torch.nn.functional.avg_pool2d(input, kernel_size=4,
14                                stride=0) # bug

```

Listing 3. Example of shared bugs in parameter-similar APIs within the DL framework.

```

1 # A bug in the torch.nn.ReflectionPad2d API.
2 import torch
3 padding_num = -8353862602220610428
4 # bug
5 torch.nn.ReflectionPad2d(padding=padding_num)
6
7 # A bug in the flow.nn.MaxPool1d API.
8 import oneflow as flow
9 padding_num = -8353862602220610428
10 pool = flow.nn.MaxPool1d(2, stride=2, return_indices
11                           =True, padding=padding_num)
12 input = flow.tensor([[[[1., 2, 3, 4, 5, 6, 7, 8]]]])
13 output, indices = pool(input) # bug

```

Listing 4. Example of shared bugs in parameter-similar APIs across DL frameworks.

calling it with a zero stride raises a runtime error stating that “non-positive stride is not supported”. However, in version 1.5, `torch.nn.functional.avg_pool2d` still crashes with a zero stride. We further observe that even in the latest versions, APIs with stride parameters, such as `torch.nn.quantized.Conv1d`, still crashes due to the same issue. This highlights two points: 1) APIs with different functionalities but similar parameters may also share bugs. 2) fixing a bug in one API does not always address similar issues in others with the same parameter, as each API may implement its functionality independently. Moreover, as shown in Listing 4, this case also shares bugs between frameworks. We classify such pairs of APIs as *parameter-similar* APIs, which share comparable input parameters, even though they may perform different operations or tasks.

The *parameter-similar* APIs serve as an important complement to *operation-similar* APIs, and we provide definitions for both types of APIs in Section III-B, along with the details of matching these two types of APIs.

III. APPROACH

We design MirrorFuzz to address the challenges of leveraging shared bugs for fuzzing. Figure 2 provides an overview of our approach, which includes three major stages: recognizing buggy APIs, matching similar APIs, and synthesizing test cases for fuzzing.

In the first step, MirrorFuzz uses web crawlers to collect GitHub issues related to DL frameworks. It filters issues based on specific bug-related keywords and employs LLM to locate buggy APIs, analyze bug causes, and identify triggering

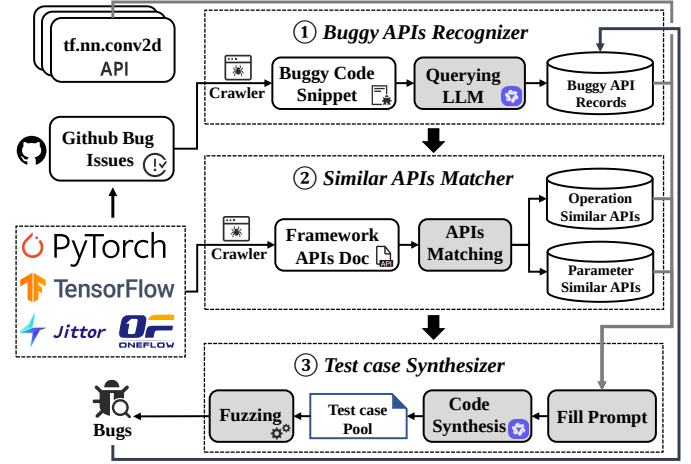


Fig. 2. MirrorFuzz Overview.

parameters. The results are then saved in buggy API records (see Section III-A). Next, MirrorFuzz extracts API metadata from the documentation. It identifies operation-similar and parameter-similar APIs within and across frameworks by measuring text and semantic similarity (see Section III-B). For an API under test, MirrorFuzz retrieves similar APIs, queries their bug records, and includes the bug data in the LLM prompt to synthesize API invocation code (see Section III-C). Finally, MirrorFuzz uses the code synthesized by the LLM as test cases for fuzz testing to enable early bug detection. The following sections explain each step in detail.

A. Buggy APIs Recognition

At this stage, MirrorFuzz uses LLM to identify buggy APIs and triggering parameters from API-related bug reports. It summarizes bug causes based on report context and saves the results in structured records.

We observe that CVE repositories often provide limited details about specific buggy APIs. However, such details are crucial for bug discovery. For instance, PyTorch has only a few bugs documented in the CVE repository [50], while a significantly larger number of bug reports are actively submitted and discussed in GitHub issues [51]. Therefore, MirrorFuzz collects API bug reports from GitHub issues.

API Bug Report Collection. To be specific, we implement a web crawler based on the GitHub REST API [52] to automatically collect issues from various DL frameworks. Since these issues include many reports unrelated to bugs (e.g., feature requests, performance improvements, help requests, and documentation issues), it is necessary to filter them. We employ a keyword-based matching method, similar to [32], to collect bug-related issues instead of relying on the commonly used “bug” labels on GitHub. This is because such labels may not correspond to the specific types of bugs we study: some may be incorrectly assigned, labeling issues unrelated to actual bugs, while others may be missing for issues that actually cause API bugs in certain frameworks. The keyword list is compiled using three strategies: (1) extracting common terms from prior DL framework testing studies [24]–[26], [29], [30],

TABLE I
STATISTICS OF COLLECTED ISSUES.

Framework	ALL Issues	Bug Issues	Code Snippets
TensorFlow	40348	1835 (4.55%)	3693
PyTorch	46486	2350 (5.06%)	3257
OneFlow	970	94 (9.69%)	141
Jittor	356	57 (16.01%)	76
Total	88160	4336 (4.92%)	7167

(2) adopting the custom keyword set from Orion [32], and (3) manually adding framework-specific terms, including Chinese terms found in bug reports for Jittor and OneFlow. The final list of selected keywords for filtering includes: “*crash*”, “*aborted (core dumped)*”, “*assertion failure*”, “*segmentation fault (core dumped)*”, “*floating point exception*”, and others. The complete list of keywords is available in our open-source code [41] and can be extended as needed.

As a result, we collected 88160 issues from the four frameworks on GitHub. Using a keyword-matching approach, we identified a total of 4336 bug-related issues (refer to Table I). Next, we use regular expressions to extract buggy code snippets, issue titles, and descriptions from these issues. For example, bug code snippets are typically marked with Markdown code tags (i.e., ``code``), which allows us to extract the code based on these tags easily. This approach may introduce false positives, as some contributors mistakenly use Markdown code blocks to represent logs, code output, etc. Thus, these code snippets are further filtered in subsequent processing. Moreover, we observe that many issues share reproducible bug code in cloud development environments (e.g., Google Colab [53]) to facilitate quick verification and debugging for developers. To address this, we employ a web crawler to collect the relevant shared links and extract the reproducible code from them. Eventually, we obtain structured data with the issue title, description, and code snippet.

Buggy APIs Recognition. After obtaining API-related bug issues, our next step is to identify the buggy APIs by analyzing the context of each issue. We do not use syntax parsers for this task, because tools such as Tree-sitter [54] can only analyze syntax. They cannot interpret the semantics of bug reports containing multiple types of information (e.g., natural language descriptions, code snippets, and error logs), determine which API causes a bug, or identify the root cause. This challenge is further compounded by the variable quality of GitHub issues, whose clarity and completeness depend on the submitters’ expertise and may lead to incomplete or unclear reports. As a result, traditional methods such as regex matching or syntax parsers struggle to reliably extract buggy APIs and identify the root causes of bugs.

To overcome this, we design an automated buggy API extraction method, utilizing prompt engineering to guide LLM in recognizing buggy APIs and parameters from the issues. We observe that the code snippets in the issues often explicitly contain the buggy API. The LLM’s understanding of code allows it to identify the buggy API from them. Specifically, we first compile the code extracted from the issue

Question:
You will analyze a code snippet to identify a buggy API call and explain the associated issue. Think step-by-step as below examples.

Example:
Issue Title: Segmentation fault (core dumped) in torch.nn.functional.max_pool1d
Issue Desc: torch.nn.functional.max_pool1d triggered a crash when stride and kernel_size are too large...
Issues Code Snippet:

```
import torch
kernel_size = 9223372036854775807
stride = 9223372036854775807
input_params = torch.randn(2, 10, 4)
output = torch.nn.functional.max_pool1d(input_params, kernel_size, stride)
...
```

Task 1: Please give a brief explanation of the root cause of this bug based on the bug report.
Task 2: Please identify the buggy API name in the code snippet that is causing the described issue.
Task 3: Please provide the list of bug-triggering parameters.
Please return the answer in strict JSON format.

LLM Example Answer as JSON Format: {
"explanation": "The root cause of the bug is that the kernel_size and stride values provided to torch.nn.functional.max_pool1d are excessively large. These values exceed the dimensions of the input tensor, leading to invalid memory access.",
"buggy_api": "torch.nn.functional.max_pool1d",
"parameters": [{"kernel_size", "stride"}]
}

Question:
Issue Title: [issue title]
Issue Desc: [issue description]
Issues Code Snippet: ``[issue code]`` Task 1 : ... Task 2 : ... Task 3 : ...

(a) Example prompt for buggy APIs recognition

Question:
You are a helpful code assistant expert in Deep Learning Framework. Based on the provided issue description and code snippet, verify whether the specified API is the one triggering the bug, and provide the verification status and a brief explanation in strict JSON format.

Example:
Issue Title: Segmentation fault (core dumped) in torch.nn.functional.max_pool1d
Issue Desc: torch.nn.functional.max_pool1d triggered a crash when stride and kernel_size are too large...
Issues Code Snippet: ``import torch ...``
Specified API: torch.nn.functional.max_pool1d

LLM Example Answer as JSON Format: {
"verification": "YES",
"explanation": "The issue arises because it is called with large kernel_size and stride values, leading to a segmentation fault..." }
...

(b) Example prompt for buggy APIs verification

Fig. 3. Prompt template for buggy APIs recognition and verification.

to filter out non-code content that uses Markdown code tags. Then, as shown in Figure 3, we use a few-shot Chain-of-Thought (CoT) [55] prompting technique to guide the LLM in identifying the buggy API based on the issue information and the code itself. The LLM may generate incomplete API names based on the API calls in the code. For example, it may identify `nn.conv2d` as the buggy API in Listing 3. While `nn.conv2d` is indeed a buggy API in this case, it is incomplete, as the expected name is `torch.nn.conv2d`. To resolve this, we calculate the Levenshtein Distance [56] between the complete API names and identified buggy API names to recognize the most similar API, ensuring the API name is complete. To further improve the accuracy of buggy API identification, we integrate the buggy APIs identified by the LLM with the issue context into a prompt, enabling the LLM to validate them.

B. Similar APIs Matching

At this stage, the matcher identifies pairs of APIs that are operation-similar and parameter-similar based on the similarity of API metadata (i.e., name, parameters, and documentation descriptions). We combine text and semantic similarity to address the challenges of matching API pairs between frameworks. Finally, to improve matching accuracy, we propose an API selection algorithm to optimize the matching results.

We first define the characteristics of these APIs and then introduce the matching process.

Definition 1 (Operation-similar APIs). Given a pair of APIs, S and T , which come from the same or different frameworks,

if S and T perform the same or similar functionalities, we define them as operation-similar (OS). Formally,

$$OS(S, T) \iff Functionality(S) \sim Functionality(T)$$

where $Functionality$ denotes an API functionality and $Functionality(S) \sim Functionality(T)$ indicates that S and T have similar functionalities. This definition is not limited to the computational operations mentioned in the motivation, such as pooling or convolution, but also covers various categories of APIs (e.g., data processing, model building, and serialization), with a primary focus on functional similarity.

Definition 2 (Parameter-similar APIs). Given a pair of APIs, S and T , which come from the same or different frameworks, if they have similar parameters and are not operation-similar, we define them as parameter-similar (PS). Formally,

$$\begin{aligned} PS(S, T) &\iff \neg OS(S, T) \\ &\quad \wedge \exists p \in Params(S) \\ &\quad \wedge \exists q \in Params(T) \\ &\quad \wedge p \sim q \end{aligned}$$

where $Params(S)$ and $Params(T)$ represent the sets of parameters for APIs S and T , respectively, and $p \sim q$ indicates that parameters p and q are similar.

Matching operation-similar and parameter-similar APIs within and across frameworks is challenging due to the diversity of APIs. Fortunately, these frameworks provide documentation and API metadata, which include details such as the API name, parameters, and functionality descriptions. For instance, with the API `torch.nn.FractionalMaxPool3d`, we can extract details from the official documentation, including its name, description (e.g., 'performs 3D fractional max pooling over an input signal consisting of multiple input planes'), and parameter information (e.g., `kernel_size`, the size of the window to take a max over). We match API pairs by calculating the similarity of their metadata, considering both text and semantic similarities. In particular, we define two functions, $Sim_{text}()$ and $Sim_{sem}()$, to measure text and semantic similarity between API S and T , respectively.

Matching Operation-similar APIs. For operation-similar APIs, we first calculate text similarity based on metadata and evaluate semantic similarity and then combine them to obtain an overall similarity score.

Specifically, for a pair of APIs S and T , given the complete metadata M^{All} (including API names, parameters, and descriptions), the text similarity between S and T can be calculated as follows:

$$\begin{aligned} Sim_{text}(M_S^{All}, M_T^{All}) = \max &\left(Sim_{text}(M_S^{name}, M_T^{name}) + \right. \\ &Sim_{text}(M_S^{param}, M_T^{param}), \\ &Sim_{text}(M_S^{desc}, M_T^{desc}) + \\ &\left. Sim_{text}(M_S^{param}, M_T^{param}) \right) \end{aligned} \quad (1)$$

Here, the max function ensures that the highest similarity score is selected from the combined text similarity of names and parameters, or descriptions and parameters. To calculate the text similarity $Sim_{text}(M_S^{All}, M_T^{All})$, we use Term Frequency-Inverse Document Frequency (TF-IDF) [57] and

cosine similarity [58]. TF-IDF measures the importance of terms based on their frequency and the uniqueness of terms in the document collection. In particular, TF indicates how often a term appears in a document, while IDF reduces the weight of common terms (such as "torch" or "tf"), thereby emphasizing specific terms that differentiate the API.

Formally, given a collection of documents D , a document $d \in D$, and a term t , the TF-IDF calculation formula is:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D) \quad (2)$$

where $TF(t, d)$ is the term frequency of t in document d , and $IDF(t, D)$ represents the inverse document frequency of term t in the document collection D . For APIs within the same framework, we construct separate documents for their name, parameters, and description metadata. Then, given a pair of APIs S and T , specific metadata M , we convert them into TF-IDF vectors $\mathbf{v}_S^M$ and $\mathbf{v}_T^M$, and then calculate their cosine similarity:

$$Sim_{text}(M_S, M_T) = \cos(\mathbf{v}_S^M, \mathbf{v}_T^M) = \frac{\mathbf{v}_S^M \cdot \mathbf{v}_T^M}{\|\mathbf{v}_S^M\| \|\mathbf{v}_T^M\|} \quad (3)$$

TF-IDF focuses solely on term occurrence frequency, overlooking semantic relationships between terms [59]. As a result, it considers semantically similar terms like "GetData" and "FetchData" to be completely different. This limitation complicates matching similar APIs across frameworks with differing naming conventions. Thus, we introduce semantic similarity to address this limitation by capturing the semantic relationships between metadata. Similarly, we calculate semantic similarity using the same Equation 1 as for text similarity. Then, we use the M3-embedding model [60] to capture semantic information from the metadata of APIs. The M3-embedding model generates embeddings where cosine distance reflects semantic similarity.

Formally, given a text X , the M3-embedding encodes it into a semantic vector $\mathbf{e}_X$:

$$\mathbf{e}_X = M3\text{-embedding}(X)$$

For calculating the semantic similarity of metadata, given a pair of APIs S and T , and specific metadata M , we first use M3-embedding model to encode the API's specific metadata to obtain the corresponding embedding vectors $\mathbf{e}_S^M$ and $\mathbf{e}_T^M$, and then calculate the similarity between these embedding vectors using cosine similarity:

$$Sim_{sem}(M_S, M_T) = \cos(\mathbf{e}_S^M, \mathbf{e}_T^M) = \frac{\mathbf{e}_S^M \cdot \mathbf{e}_T^M}{\|\mathbf{e}_S^M\| \|\mathbf{e}_T^M\|} \quad (4)$$

Finally, we consider both text and semantic similarity, with the calculation for operation-similar APIs as follows:

$$\begin{aligned} OS(S, T) = &\alpha \cdot Sim_{text}(M_S^{All}, M_T^{All}) \\ &+ (1 - \alpha) \cdot Sim_{sem}(M_S^{All}, M_T^{All}) \end{aligned} \quad (5)$$

where α is a weight parameter used to balance the contributions of text and semantic similarity.

Matching Parameter-similar APIs. We also calculate text and semantic similarity to identify parameter-similar API pairs. Note that we only match within the set of API pairs

Algorithm 1: API Selection Optimization

Input: source API S , target API list L , top- k K , threshold H

Output: list of selected similar APIs R

```

1  $R \leftarrow \{\}$ ;
2  $sim\_list \leftarrow \{\}$ ;
3 foreach  $T \in L$  do
4    $sim\_score \leftarrow \text{CalculateSimilarity}(S, T)$ ;
5    $sim\_list \leftarrow sim\_list \cup \{(T, sim\_score)\}$ ;
6  $sim\_list \leftarrow \text{SortSimDesc}((sim\_list))$ ;
7  $R \leftarrow sim\_list[0 : K]$ ;
8 foreach  $(T, sim\_score) \in sim\_list[K : ]$  do
9   if  $sim\_score \geq H$  then
10     $R \leftarrow R \cup \{(T, sim\_score)\}$ ;
11 return  $R$ ;
```

that are not operation-similar, which helps avoid redundant matches. Similar to operation-similar APIs, we use TF-IDF to calculate text similarity and the M3-embedding model to calculate semantic similarity. Notably, $PS(S, T)$ relies on only one metadata, allowing direct calculation of similarity without adhering to Equation 1. Therefore, given a pair of APIs S and T , along with their parameter metadata M^{param} , the calculation for parameter-similar APIs is as follows:

$$PS(S, T) = \alpha \cdot Sim_{text}(M_S^{Param}, M_T^{Param}) + (1 - \alpha) \cdot Sim_{sem}(M_S^{Param}, M_T^{Param}) \quad (6)$$

where a weight parameter is the same as the definition of operation-similar APIs.

API Selection Optimization. Each source API has a different number of similar APIs, and the distributions of their similarity scores can vary across frameworks. The widely used Top- k method [25], [61] has limitations: a small k may miss valuable APIs, while a large k increases fuzzing overhead. Similarly, a fixed threshold H can also be problematic due to differences across frameworks. To address these, we propose Algorithm 1, a two-stage filtering strategy combining Top- k selection with a similarity threshold H , instead of relying solely on either Top- k or threshold-based methods. First, it calculates the text and semantic similarity between source API S and all target APIs L (line 4). Then, it selects the top k APIs with the highest similarity scores (line 7). Additionally, it applies threshold filtering to the remaining APIs. Those ranked $k + 1$ and beyond are included if their similarity exceeds the threshold (line 9). In summary, Top- k ensures a minimum set of high-quality matches, while the threshold H further supplements this set by including additional relevant APIs beyond k . When the similarity scores of the Top- k APIs are all greater than or equal to H , the algorithm effectively reduces to a purely threshold-based strategy. We set the default Top- k value to 6, which may occasionally include a few irrelevant APIs. However, the powerful reasoning capabilities of modern LLMs can handle these cases, and the additional overhead is considered acceptable in the overall fuzzing process. The threshold H is determined through empirical observation and

is primarily influenced by the scope of API matching. Specifically, we assign different thresholds for within-framework and cross-framework matching, since even functionally similar APIs across frameworks can differ significantly in naming conventions and documentation. Notably, the threshold H and the value of k are not fixed and can be adjusted by users or researchers to suit different requirements and practical scenarios.

C. Test Case Synthesis

In this stage, MirrorFuzz generates test cases for the API under test by leveraging an LLM to analyze the root causes of bugs in similar APIs. These test cases are carefully crafted to expose analogous bugs in the API under test. Then, they are stored in the initial test case pool for subsequent fuzz testing.

The process of code synthesis is critical, as it must account for the complex input constraints and parameter dependencies of various framework APIs. Although bug patterns may be shared, the test code itself cannot be directly reused. This is because even functionally similar APIs may differ in naming conventions, parameter specifications, or framework-specific constructs. Bugs may also arise from diverse root causes, including invalid API call sequences, tensor shape mismatches, or data type incompatibilities. As a result, simply migrating the inputs that caused a bug in one API may produce invalid test cases and fail to trigger bugs in the target API.

Recent studies [30], [31] have demonstrated that LLMs, trained on extensive corpora containing code snippets for DL framework APIs, can effectively understand the syntax and semantics of these APIs. This capability enables LLMs to generate valid API invocation codes suited to specific DL frameworks. Therefore, MirrorFuzz leverages the reasoning and code generation capabilities of LLMs to synthesize code, which is then used as test cases for fuzz testing. To achieve this, MirrorFuzz retrieves buggy API records from APIs that are operation-similar and parameter-similar to the API under test. These records include details such as buggy APIs, problematic parameters, relevant code snippets, and the root causes of the bugs. Using this information, MirrorFuzz guides the LLM to create test cases designed to trigger similar bugs in the API under test.

Code Synthesis. Specifically, the code synthesis process is as described in Algorithm 2. We first query APIs that are similar in operations and parameters to the API S , as well as the documentation information for API S , and initialize T to store the test cases (line 1-4). Next, we iterate through the list of APIs with operations and parameters similar to API S and query their bug data from the bug records (line 5-7). After querying the bug data, we design a prompt (i.e., *syn_prompt*) for code synthesis, allowing the LLM to generate test cases for API S based on the queried bug data (line 9-10). We execute the synthesized code to verify that the test cases are executable (line 11). If syntax or runtime errors occur, we use the LLM to attempt fixes based on the error messages (line 12-15). This process deals with exceptions in LLM-synthesized code. When a synthesized test case fails due to syntax or runtime errors (e.g., violations of API constraints),

Algorithm 2: Test case Synthesis

```

Input: API under test  $S$ 
Output: test case pool for the API under test  $T$ 
1  $oper\_list \leftarrow \text{QueryOperationSimAPIs}(S)$ ;
2  $para\_list \leftarrow \text{QueryParameterSimAPIs}(S)$ ;
3  $S\_doc \leftarrow \text{QueryAPIDoc}(S)$ ;
4  $T \leftarrow \{\}$ ;
5 foreach  $sim\_api, sim\_type \in (oper\_list, para\_list)$  do
6    $bug\_list \leftarrow \text{QueryBugRecords}(S, sim\_api)$ ;
7   foreach  $bug\_data \in bug\_list$  do
8     //  $bug\_data$  includes buggy APIs, parameters, code, and root cause.
9      $syn\_prompt \leftarrow$ 
10       $\text{FillPrompt}(S, S\_doc, bug\_data, sim\_type)$ ;
11      $test\_case \leftarrow \text{LLM}(syn\_prompt)$ ;
12      $status, out \leftarrow \text{exec}(test\_case)$ ;
13     if  $status \in \text{Error}$  then
14        $f\_status \leftarrow$ 
15         $\text{FixbyLLM}(test\_case, status, out)$ ;
16       if  $f\_status == \text{False}$  then
17         continue;
18      $score \leftarrow \text{Priority}(test\_case)$ ;
19      $T \leftarrow T \cup \{(test\_case, score)\}$ ;
20 return  $T$ ;

```

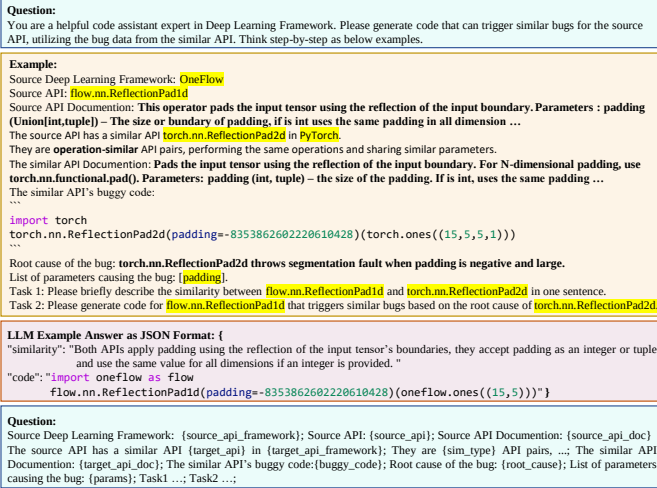


Fig. 4. Prompt template for test case synthesis.

we provide the LLM with the exception information to repair it, giving the test case additional chances to meet syntactic and semantic requirements while preserving its potential to expose shared bugs. Finally, we evaluate the quality of the test cases and save them into the test case pool T (line 16-17). For syn_prompt , we employ the few-shot CoT prompting technique, as illustrated in Figure 4.

Fuzzing Executor. Then, MirrorFuzz selects test cases from the test case pool, applies mutation operations, and executes them to detect potential bugs in DL frameworks. It identifies bugs by monitoring crashes within the framework. Furthermore, for newly discovered bugs during fuzz testing, MirrorFuzz updates its bug records to identify more potential shared bugs across APIs.

For test case selection, we rank each synthesized test case

T for API S using the following formula:

$$\text{Priority}(T) = G \cdot (U - C) \quad (7)$$

Here, T represents a synthesized test case, and $\text{Priority}(T)$ quantifies its ranking score. The binary flag G ensures that the generated code includes the API S , preventing the hallucination issue in LLMs that may generate invalid test cases [62]. U represents the number of unique APIs covered by the test case, aiming to explore diverse API combinations. C represents execution time, which helps prevent long-running test cases from consuming excessive resources. We select the top 10 test cases with the highest priority scores for subsequent mutation operations.

For the mutation strategy, we randomly select parameters of the API under test and apply three types of mutation operators until the desired number of test cases is generated. These operators include: 1) boundary value mutation, which randomly inserts boundary values (e.g., NaN, negative values, extreme numerical values) into the input data; 2) type mutation, which randomly changes the data types of parameters, such as converting `tf.int32` to `tf.float16`; and 3) shape and dimension mutation, which modifies input shapes and sizes, such as reshaping tensors.

For test oracle, we detect bugs by monitoring framework crashes, such as program aborts, segmentation faults, and floating-point exceptions. Additionally, we observe that certain internal errors do not result in crashes but instead cause the framework to halt, often accompanied by messages such as, "Please report this bug to the relevant platform/organization." Therefore, we include these cases in our monitoring process to ensure they are properly recorded and reported. Since Jittor [39] integrates an operator compiler and tuner, it compiles the entire framework and meta-operators just-in-time. As a result, we also detect bugs where Jittor encounters compilation failures or internal failures.

IV. EVALUATION

A. Implementation

We implement MirrorFuzz in Python and select commonly used DL Frameworks, including PyTorch (v2.4.1) [13], TensorFlow (2.17.0) [14], OneFlow (v0.9) [38], and Jittor (v1.3.9.5) [39], all of which have been previously studied in DL framework testing research [23], [28], [31]. It utilizes the bs4 library [63] to parse the API documentation of DL frameworks, supporting the API information needs of its three modules. The buggy API recognition module is developed using the requests library [64] to scrape GitHub issues. The similar API matching module is implemented based on M3-embedding [60], which is employed to obtain semantic embeddings for API pairs. We set α to 0.35, prioritizing semantic similarity to address the challenges of cross-framework matching. We then retain the Top-6 similar APIs. For the LLM, we utilize Qwen2.5-Coder-7B-Instruct-AWQ [47] for test case synthesis. Qwen2.5-Coder represents the latest series of code-specific Qwen LLMs, offering significant improvements in code generation, code reasoning, and code fixing. We obtain the model files from Hugging Face [65] and deploy them locally.

TABLE II
LLMS EVALUATED IN MIRRORFUZZ.

LLM	Model	Quantization	Generation Speed	Size
CodeLLama	7B-Instruct	4-bit	15.03 tokens/s	3.89GB
CodeGemma	7B-IT(Instruct Tuned)	4-bit	12.23 tokens/s	6.68GB
Qwen2.5-Coder	7B-Instruct	4-bit	13.05 tokens/s	5.57GB

B. Experimental Setup

Baseline Selection. Our approach is an API-level fuzzer, so we focus on state-of-the-art API-level fuzzers, including TENSORSCOPE [9], TitanFuzz [30], Orion [32], FuzzGPT [31], and Future [35]. To gain a comprehensive understanding, we also consider model-level fuzzers such as MoCo [40]. Among these, FuzzGPT and Future are promised to be open-source. However, as of the completion of our work, their code is still not publicly available. Therefore, we choose TENSORSCOPE, TitanFuzz, Orion, and MoCo as our baselines.

DL Framework. Since some baselines do not support OneFlow and Jittor, we mainly conduct evaluations on TensorFlow and PyTorch. Except for the DL Framework versions implemented in MirrorFuzz, we also conduct bug detection for each baseline on PyTorch (v1.31.1) and TensorFlow (2.10.0) to ensure a fair comparison, as these versions are the same as or closest to those used by most baselines, minimizing version differences.

Large Language Models. To comprehensively evaluate the performance of MirrorFuzz in synthesizing shared bugs using LLMs, we selected Qwen2.5-coder [47], CodeGemma [46], and CodeLlama [66], deploying them in a local environment. The specifications of the LLMs used in our evaluation are listed in Table II.

Environment. We use a 64-core workstation with 256 GB RAM, running Ubuntu 22.04 LTS, and equipped with 2 NVIDIA RTX 4090 GPUs.

C. Evaluation Metrics.

Number of covered APIs. We report the number of API coverages, as there are a large number of APIs in the DL framework.

Code coverage. Code coverage is a widely used test adequacy metric in testing DL frameworks. We use the coverage.py [67] library to report the number of covered Python code lines in the DL framework.

Number of detected bugs. Following prior work on fuzzing DL Frameworks [24], [25], [30], [31], we report the number of detected bugs.

Validity and success rate of synthesis. MirrorFuzz leverages LLMs to synthesize initial test cases. We evaluate the synthesis validity rate via JSON parsing and Python syntax detection, as well as the success rate of LLMs in reproducing known bugs in similar APIs.

V. RESULTS ANALYSIS

We evaluate MirrorFuzz by answering the following four research questions:

- **RQ1:** How do different prompt strategies affect the accuracy of recognizing buggy APIs from bug reports?
- **RQ2:** How prevalent are similar APIs across DL frameworks, and how effectively can MirrorFuzz identify them?
- **RQ3:** How effective is MirrorFuzz in synthesizing shared bugs and detecting real-world bugs?
- **RQ4:** How does MirrorFuzz perform relative to state-of-the-art approaches?

A. Accuracy of Buggy APIs Recognition

In this section, we conduct an ablation study to evaluate the effectiveness of the prompting strategy for buggy API recognition. We randomly sample 280 issues from the crawled GitHub issues for four frameworks, with the three authors manually identifying buggy APIs and bug-causing parameters, followed by cross-validation to ensure accuracy. We design four prompt variants: “#Issue w/o T” (without issue title), “#Issue w/o D” (without description), “#Issue w/o TD” (without both), and “#Issue w/ ALL” (with code, title, and description). These variants are evaluated under two conditions: few-shot CoT and zero-shot, using the latter as the baseline. To mitigate LLM randomness, we repeat each experiment independently 10 times per framework and average the results. As shown in Table III, providing buggy API recognition examples, complete issue context, and refining the task through few-shot CoT techniques improves the LLM’s performance, achieving 95.23% accuracy in buggy API recognition and 75.83% in bug-causing parameter identification. This demonstrates that our method effectively identifies buggy APIs and bug-causing parameters from GitHub issues and is effective across multiple frameworks.

Furthermore, we observe that in OneFlow, “#Issue w/o T” achieves the highest accuracy in bug-causing parameter recognition, while “#Issue w/o D” excels in buggy API recognition. In Jittor, “#Issue w/o D” achieves the highest accuracy in bug-causing parameter recognition. This may stem from the fact that issue reports submitted in OneFlow and Jittor often fail to meet community standards, resulting in generally lower-quality reports compared to those of TensorFlow and PyTorch. For instance, many descriptions are vague and lack clarity, failing to provide the root causes of the reported issues. Such issues can hinder LLMs’ ability to analyze and handle them effectively.

B. Extent of API Similarity in DL Frameworks

To uncover API commonalities across frameworks, we use our method to quantify operation-similar (#OS) and parameter-similar (#PS) APIs within and across frameworks. As shown in Table IV, each row and column represents a framework, with rows as the source and columns as the target. For example, TensorFlow has 2408 #OS and 737 #PS API pairs in PyTorch, while PyTorch has 2509 and 609 in TensorFlow. The table reveals numerous similar API pairs across frameworks, indicating that frameworks are widely vulnerable to shared bugs. Notably, TensorFlow and PyTorch have the most matching API pairs, primarily because they offer more APIs than Jittor and OneFlow. Overall, #PS API pairs are fewer than #OS

TABLE III
EXPERIMENTAL RESULTS OF BUGGY APIs RECOGNITION IN DIFFERENT PROMPT STRATEGIES.

Framework	Codes	Approach	#Issue w/o T		#Issue w/o D		#Issue w/o TD		#Issue w/ ALL	
			APIs Acc	Param Acc	APIs Acc	Param Acc	APIs Acc	Param Acc	APIs Acc	Param Acc
TensorFlow	100	Zero-shot	91.20%	59.30%	92.80%	58.80%	90.80%	49.80%	93.80%	64.60%
		Few-shot CoT	96.50%	63.00%	95.30%	60.70%	93.80%	55.60%	96.70%	66.80%
PyTorch	100	Zero-shot	75.00%	65.60%	79.90%	66.90%	69.90%	54.70%	80.70%	68.30%
		Few-shot CoT	87.40%	68.30%	91.40%	69.20%	83.40%	63.20%	92.10%	71.00%
OneFlow	50	Zero-shot	90.80%	76.70%	91.00%	72.70%	88.40%	66.90%	91.80%	77.80%
		Few-shot CoT	92.20%	78.60%	94.20%	72.90%	92.20%	68.60%	92.40%	75.50%
Jittor	30	Zero-shot	94.00%	88.00%	99.00%	90.00%	99.30%	93.30%	90.00%	87.30%
		Few-shot CoT	99.00%	89.30%	99.70%	94.70%	99.70%	91.70%	99.70%	90.00%
Total	280	Zero-shot	87.75%	72.40%	90.68%	72.10%	87.10%	66.18%	89.08%	74.50%
		Few-shot CoT	93.78%	74.80%	95.15%	74.38%	92.28%	69.78%	95.23%	75.83%

TABLE IV
MATCHED OPERATION-SIMILAR (#OS) AND PARAMETER-SIMILAR (#OP) API PAIRS WITHIN AND ACROSS DL FRAMEWORKS.

	TensorFlow		PyTorch		OneFlow		Jittor	
	#OS	#PS	#OS	#PS	#OS	#PS	#OS	#PS
TensorFlow	19980	7467	2408	737	1277	227	800	420
PyTorch	2509	609	4502	2355	6310	937	2252	807
OneFlow	1269	73	6063	571	714	144	623	153
Jittor	808	539	2275	841	628	234	1357	178
Total	24566	8688	15248	4504	8929	1542	5032	1558

TABLE V
EVALUATION OF API MATCHING METHODS UNDER DIFFERENT SETTINGS.

	Matcher w/o optimization		Matcher w/ optimization	
	#Within	#Cross	#Within	#Cross
TensorFlow	63.52%	89.79%	80.85%	91.27%
PyTorch	66.33%	78.39%	82.99%	89.90%
OneFlow	85.94%	70.60%	89.06%	76.54%
Jittor	88.69%	81.48%	92.26%	86.48%
Total	76.12%	80.07%	86.29%	86.05%

ones. However, APIs with different names and functionalities may share parameter designs, potentially causing shared bugs. Thus, #PS pairs are a crucial complement to #OS ones.

Additionally, we randomly select 40 APIs (10 from each of four frameworks) and ask three authors familiar with DL frameworks to manually label similar API pairs within and across frameworks. To evaluate the effectiveness of our method, we calculate the coverage rate of the labeled API pairs identified by our method. In total, we manually label 596 #OS API pairs for 40 APIs, including 156 within-framework (“#Within”) pairs and 440 cross-framework (“#Cross”) pairs. We conduct experiments both with and without API selection optimization (Algorithm 1). “Matcher w/ optimization” uses Top- k selection combined with a similarity threshold, while “Matcher w/o optimization” uses only Top- k selection. As shown in Table V, the default setting “Matcher w/ optimization” achieves average matching coverage rates of 86.29% within frameworks and 86.05% across frameworks, outperforming the “Matcher w/o optimization” setting. This demonstrates that our method can effectively match similar

APIs both within and across frameworks, and that the API selection optimization further enhances the results.

C. Effectiveness of Synthesis and Detected Bugs

Validity and Success Rate of Synthesis. To evaluate the effectiveness of MirrorFuzz in synthesizing shared bugs, we constructed a benchmark consisting of 20 shared bugs: 10 for #OP APIs and 10 for #PS APIs, covering 4 frameworks. We use a zero-shot approach as the baseline, where LLMs directly synthesize shared bugs based on the provided API pairs and bug code. Each model has three opportunities to synthesize. Table VI shows the validity rate and success rate of synthesizing shared bugs for MirrorFuzz and the baseline. The few-shot CoT prompt, constructed by MirrorFuzz, achieves a 100% validity rate for the synthesized shared bugs under Qwen2.5-Coder. This means that the generated code passes both JSON parsing and Python syntax checks. Moreover, it successfully reproduces 17 shared bugs, achieving a 100% success rate for #OS APIs, outperforming both the baseline and other models. This demonstrates that our method enables LLMs to effectively synthesize shared bugs. Furthermore, Qwen2.5-Coder exhibits superior code understanding and generation capabilities, further enhancing its performance. Therefore, MirrorFuzz uses Qwen2.5-Coder by default.

Real-world Bugs Detected. Table VII summarizes real-world bugs detected by MirrorFuzz across four popular DL frameworks. The “Total” column indicates the total number of bugs detected by MirrorFuzz, while the “Unknown” and “Known” columns show the number of previously unknown and previously known bugs, respectively. Following [25], [28], we report the number of bugs confirmed by developers as previously unknown (“Confirmed” column); the number of bugs rejected by developers (“Rejected” column); and the number of previously unknown bugs that have been fixed (“Fixed” column). Additionally, we categorize bugs by type, including segmentation faults (“Segv”), floating-point exceptions (“FPE”), program aborts (“Abort”), and framework internal failures (“Other”).

Overall, MirrorFuzz detects 315 bugs, of which 262 are previously unknown. Among these, 180 are confirmed by developers, 80 are fixed, and 9 are rejected. Among the rejected bugs, 4 are due to developers being unable to reproduce

TABLE VI
SYNTHESIS VALIDITY AND SUCCESS RATES OF SHARED BUGS. #OS DENOTES OPERATION-SIMILAR, AND #PS DENOTES PARAMETER-SIMILAR.

	LLM	Validity Rate			Success Rate		
		#OS	#PS	ALL	#OS	#PS	ALL
Zero-shot	CodeLLama	66.67%(20/30)	33.33%(10/30)	50.00%(30/60)	40.00%(4/10)	30.00%(3/10)	35.00%(7/20)
	CodeGemma	76.67%(23/30)	70.00%(21/30)	73.33%(44/60)	40.00%(4/10)	60.00%(6/10)	50.00%(10/20)
	Qwen2.5-Coder	100.00%(30/30)	93.33%(28/30)	96.67%(58/60)	50.00%(5/10)	80.00%(8/10)	65.00%(13/20)
MirrorFuzz	CodeLLama	73.33%(22/30)	56.67%(17/30)	65.00%(39/60)	60.00%(6/10)	40.00%(4/10)	50.00%(10/20)
	CodeGemma	93.33%(28/30)	83.33%(25/30)	88.33%(53/60)	60.00%(6/10)	70.00%(7/10)	65.00%(13/20)
	Qwen2.5-Coder	100.00%(30/30)	100.00%(30/30)	100.00%(60/60)	100.00%(10/10)	70.00%(7/10)	85.00%(17/20)

TABLE VII
BUGS FOUND BY MIRRORFUZZ. FOR THE BUG TYPE, “SEGV” STANDS FOR SEGMENTATION FAULT, “FPE” IS FOR FLOATING POINT EXCEPTION, AND “ABORT” MEANS PROGRAM ABORT. “OTHER” INCLUDES INTERNAL FAILURES AND COMPILATION FAILURES.

	Version	Total Bugs			Status			Type			
		Total	Unknown	Known	Confirmed	Rejected	Fixed	Segv	FPE	Abort	Other
TensorFlow	2.16 & 2.17 & nightly	118	83	35	83	7	20	11	1	106	0
PyTorch	2.3.0 & nightly	48	47	1	45	2	17	15	10	13	10
OneFlow	0.9 & 1.0	86	77	9	28	0	28	12	2	72	0
Jittor	1.3.9.5 & 1.3.9.10	63	55	8	24	0	15	1	0	6	56
Total		315	262	53	180	9	80	39	13	197	66

our proof-of-concept, 3 are due to incorrect or unsupported API usage (classified by developers as user errors rather than framework bugs), and 2, though reported for the first time, cannot be reproduced in the latest nightly builds. Across all bug types, we observe 39 segmentation faults, 13 floating-point exceptions, 197 program aborts, and 66 framework internal errors. Most bugs in Jittor fall under the “Other” category, such as compilation and internal failures, due to its just-in-time compilation.

We further evaluate bug detection capability by enabling AddressSanitizer (ASAN) [68] on TensorFlow and PyTorch, while builds for OneFlow and Jittor fail due to compilation issues. Using ASAN, we identify 9 NULL-pointer dereferences (1 memory write, 8 memory reads) and 1 heap buffer overflow in TensorFlow, whereas in PyTorch we find 8 NULL-pointer dereferences (2 memory writes, 6 memory reads), 13 heap buffer overflows, 1 container overflow, and 1 stack overflow. These findings demonstrate that MirrorFuzz is capable of detecting multiple types of bugs, including out-of-bounds reads, which may lead to severe consequences such as memory corruption or denial of service. Furthermore, 52 bugs are assigned CNVD IDs, and 5 bugs are marked as high-priority by PyTorch developers for urgent resolution.

Source Distribution of Bugs. As shown in Table VIII, we manually analyze the bugs detected by MirrorFuzz to identify their origins, specifically which frameworks’ operation-similar (#OS) and parameter-similar (#PS) APIs are responsible. We observe that bugs in these frameworks affect each other, and most of these bugs are influenced by flaws in TensorFlow and PyTorch. This is mainly due to TensorFlow and PyTorch being more comprehensive frameworks with more APIs and historical issues. Interestingly, bugs in Jittor and OneFlow can also impact TensorFlow and PyTorch, despite having fewer APIs and known issues. This highlights that even smaller or

TABLE VIII
SOURCE DISTRIBUTION OF BUGS. TF STANDS FOR TENSORFLOW, PT FOR PYTORCH, OF FOR ONEFLOW, AND JT FOR JITTOR.

	Bugs	TF		PT		OF		JT		Total		
		#OS	#PS	#OS	#PS	#OS	#PS	#OS	#PS	#OS	#PS	Total
TF	118	76	32	9	4	7	3	8	1	100	40	140
PT	48	15	5	25	6	4	1	6	3	50	15	65
OF	86	34	8	25	9	22	9	27	5	108	31	139
JT	63	43	10	38	9	28	5	26	9	135	33	168
Total	315	168	55	97	28	61	18	67	18	393	119	512

newer frameworks can impact mainstream ones, emphasizing the prevalence of shared bugs. Thus, we should focus not only on bugs within individual frameworks but also on threats arising between framework similarities. Additionally, #OS APIs are the primary source of bugs, while #PS APIs supplement gaps that #OS APIs cannot cover. Moreover, we find that some bugs are repeatedly triggered by shared bugs across different frameworks, possibly because developers make the same mistakes when implementing these features. **Case Study.** In the following paragraphs, we discuss bugs from #OS and #PS APIs, missed by previous fuzzers but found by MirrorFuzz.

Shared Bug in Operation Similarity. `torch.nn.Conv2d` is a fundamental component for constructing deep neural networks. Before PyTorch version 1.2, an issue existed where an improper configuration of the `stride` parameter could cause the framework to crash. This issue was later resolved. However, by leveraging the #OS relationships between these APIs, MirrorFuzz successfully synthesized code for the `torch.nn.quantized.Conv2d` and `torch.nn.quantized.Conv` series of APIs, revealing similar crash-inducing bugs. Developers responded promptly and addressed the reported issue.

TABLE IX

COMPARISON OF API AND CODE COVERAGE BETWEEN TENSORSCOPE, ORION, TITANFUZZ, AND MIRRORFUZZ.

	TensorFlow		PyTorch	
	API Cov	Code Cov	API Cov	Code Cov
Total	4403	284510(100%)	1992	275822(100%)
TENSORSCOPE	1469	81202(28.54%)	-	-
Orion	4037	84047(29.54%)	1751	27130(9.84%)
TitanFuzz	2215	92670(32.57%)	1329	39207(14.21%)
MirrorFuzz	3995	129663(45.57%)	1868	77707(28.17%)

TABLE X

COMPARISON OF API AND CODE COVERAGE BETWEEN MoCo AND MIRRORFUZZ BASED ON THE APIS SUPPORTED BY MoCo.

	TensorFlow		PyTorch		Jitter	
	API Cov	Code Cov	API Cov	Code Cov	API Cov	Code Cov
Total	169	284510(100%)	144	275822(100%)	72	13166(100%)
MoCo	95	74756(26.27%)	72	26271(9.52%)	66	3955(30.04%)
MirrorFuzz	145	83875(29.48%)	142	67322(24.40%)	68	4841(36.77%)

Shared Bug in Parameter Similarity. In TensorFlow, various optimizers are provided for users to choose from, helping to adjust and update model parameters, such as `tf.raw_ops.ResourceApplyAdam` and `tf.raw_ops.ResourceSparseApplyFtrlV2`. These APIs lack data type checks for the *accumulator* parameters, which can lead to runtime crashes. MirrorFuzz identifies one such crash and updates it in the bug records. Subsequently, by leveraging the relationship of #OS between their APIs, after multiple fuzzing iterations, we discover 29 similar optimizer bugs in TensorFlow.

D. Comparison with Prior Work

We compare MirrorFuzz with state-of-the-art API-level and model-level fuzzers. Despite our best efforts, we fail to reproduce TENSORSCOPE's PyTorch fuzzer, so we do not report its API coverage, code coverage, and bug detection results on PyTorch. Additionally, we use data from FreeFuzz [24] and Docter [26] to support Orion's performance.

API and Code Coverage. Table IX summarizes the API and code coverage achieved by all API-level techniques on both TensorFlow and PyTorch. MirrorFuzz demonstrates comprehensive coverage, supporting 3995 APIs in TensorFlow and 1868 APIs in PyTorch. When compared to the best-performing baseline, Orion, MirrorFuzz improves PyTorch's API coverage by 6.68%, while maintaining comparable coverage for TensorFlow. In terms of code coverage, MirrorFuzz significantly outperforms the best baseline, TitanFuzz, with a 39.92% improvement for TensorFlow and a substantial 98.20% increase for PyTorch. Since MoCo is a model-level fuzzer, we compare it only on the APIs it supports to ensure a fair comparison. As MoCo also supports Jitter, we include it in the comparison as well. Table X presents the comparison between MirrorFuzz and MoCo. In terms of API and code coverage, MirrorFuzz outperforms MoCo in both aspects. These improvements come from our carefully designed prompt templates, which use shared bugs and few-shot CoT to guide the LLM in generating more diverse and realistic code snippets. This facilitates

TABLE XI

COMPARISON OF BUG DETECTION TIME EFFICIENCY ON 400 APIS AMONG ORION, TITANFUZZ, AND MIRRORFUZZ.

	TensorFlow			PyTorch		
	APIs Tested	Bug Found	Testing Duration	APIs Tested	Bug Found	Testing Duration
Orion	148/400	14	5h	172/400	6	5h
TitanFuzz	287/400	2	5h	231/400	2	5h
MirrorFuzz	385/400	27	5h	400/400	11	4h23min44s

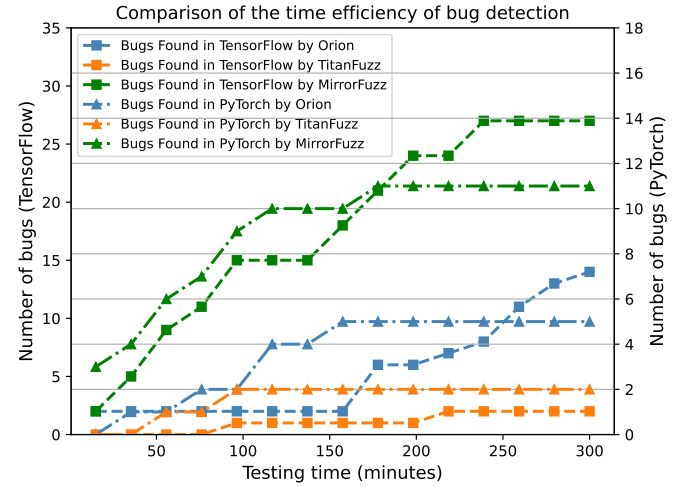


Fig. 5. Comparison of bug detection time efficiency on 400 APIs.

complex API interactions, covering essential processes such as model building, training, and serialization, enabling deeper framework exploration.

Bug Detection. We compare the crash detection time efficiency of MirrorFuzz against baseline methods. Note that we exclude inconsistency bugs, as crashes are easier to measure and serve as a good approximation of bug-finding capabilities [30]. Following the approach of TensorScope, we randomly select 400 APIs each from TensorFlow and PyTorch. We then ran each baseline tool with its default settings for a fixed duration of 5 hours. Since TensorScope only supports low-level APIs in TensorFlow, and MoCo supports a limited number of APIs, they are excluded from the efficiency comparison. As shown in Table XI, within the same time budget, MirrorFuzz tests 385 APIs in TensorFlow and completes testing all 400 APIs in PyTorch, outperforming other baselines. Figure 5 further illustrates the dynamics of bug discovery of the different tools over time. MirrorFuzz also reports the most unique bugs, finding 27 in TensorFlow and 11 in PyTorch. This demonstrates the advantage of MirrorFuzz in terms of bug detection efficiency.

Furthermore, We compare the crash detection performance of MirrorFuzz and the baselines across different versions of TensorFlow and PyTorch. Specifically, we conduct bug detection with each baseline's default settings and its respective default time budget. Figure 6 shows a comparison of MirrorFuzz and state-of-the-art fuzzers in terms of the number of bugs detected across different versions of TensorFlow and

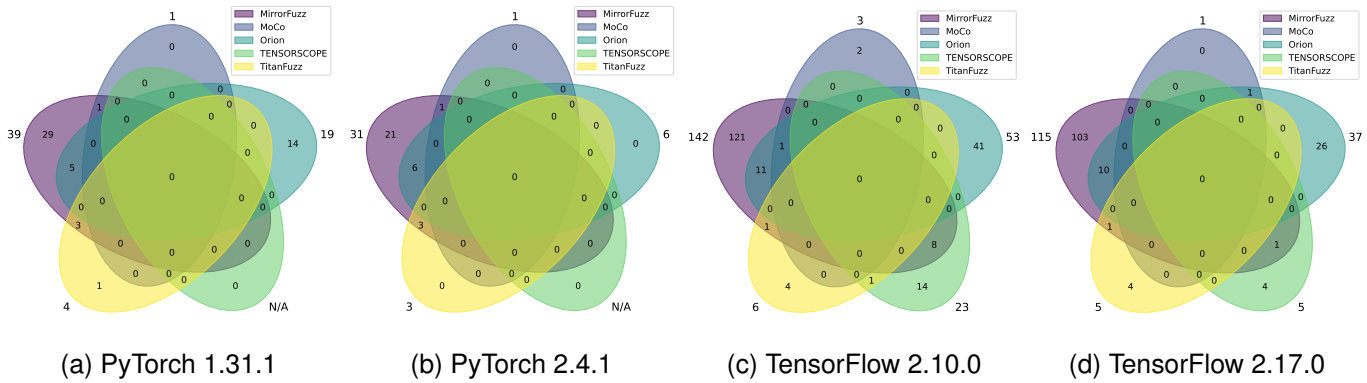


Fig. 6. Venn diagram of unique crashes on TensorFlow and PyTorch releases.

PyTorch. MirrorFuzz detects 39 and 31 bugs in PyTorch versions 1.13.1 and 2.4.1, respectively, and identifies 142 and 115 bugs in TensorFlow versions 2.10.0 and 2.17.0. Compared to the baselines, MirrorFuzz demonstrates superior performance by uncovering the highest number of crashes in both the earlier and current versions of TensorFlow and PyTorch. Remarkably, despite the extensive prior testing of these frameworks, MirrorFuzz uncovers a significant number of new bugs. This success is attributed to the shared bug phenomenon discovered in our work, enabling MirrorFuzz to synthesize code that triggers similar bugs across analogous APIs within and between frameworks, thereby revealing previously undetected flaws.

VI. THREATS TO VALIDITY

Internal. In our approach, we employ embedding models to extract semantic features from API metadata, and leverage LLMs for buggy API recognition and test case generation. Differences between frameworks can introduce semantic ambiguity, which may limit the effectiveness of embedding models. To address this issue, we simultaneously consider text and semantic similarity and design an optimization algorithm that combines different thresholds to mitigate this effect. Given the inherent unpredictability of LLMs, we strictly preprocess the crawled data and carefully design few-shot CoT prompt templates to enhance their accuracy and reliability in two distinct tasks. Another internal threat is the construction of the ground-truth. To ensure its reliability and accuracy, we assign three authors to handle the process for each build and conduct cross-validation.

External. The threats to the external validity of MirrorFuzz come from the tested DL frameworks. We select four commonly used DL frameworks, i.e., TensorFlow, PyTorch, OneFlow, and Jittor, as our research subjects. Moreover, our fuzzing approach is generalizable to other frameworks. The structure of documentation pages varies across different frameworks. Users only need to adapt the provided crawler code to match the target documentation format in order to collect API metadata. We provide a flexible GitHub issue crawler, and subsequent buggy API recognition, similar API matching, and test case synthesis are automatic and universal.

VII. RELATED WORK

To ensure the reliability of upper-layer applications built on DL frameworks, many studies [10], [11], [17], [69] reveal and analyze bugs within these frameworks. Furthermore, several fuzzing techniques are proposed to address these issues, aiming to enhance the security of the frameworks. These techniques are categorized into model-level fuzzing [7], [19]–[23], [27], [70], [71] and API-level fuzzing [8], [9], [24]–[26], [28]–[33], [72], depending on the type of test cases provided to the DL framework.

Model-level Fuzzing. Model-level fuzzing detects bugs in the framework implementation by using a complete DL model as input. These DL models consist of a range of APIs that cover various neural network functions. CRADLE [19] is a pioneering work in performing fuzzing across DL frameworks, using differential testing to detect and locate bugs by running pre-trained models across different DL backends supported by Keras [73], such as TensorFlow [14]. AUDEE [21] adopts search-based strategies and conducts cross-reference checks to detect behavioral inconsistencies across multiple DL frameworks. LEMON [20] and Muffin [22] propose various mutation strategies that further augment CRADLE to generate diverse DL models. Later, these studies [23], [27], [70], [71] propose different strategies to generate diverse DL models for bug detection in frameworks by differential testing. Recently, DeepConstr [7] refines API constraints by evaluating whether the complementary set of constraints generates valid test cases. In contrast, MirrorFuzz leverages LLM to synthesize test cases for APIs, enabling coverage of a broader range of APIs. By referencing real-world buggy code snippets, MirrorFuzz generates diverse API sequences, facilitating deeper testing of DL frameworks.

API-level Fuzzing. Compared to model-level fuzzing, API-level fuzzing focuses on detecting bugs within individual APIs. However, the complex constraints inherent in APIs pose a challenge for generating arbitrary test cases. Consequently, these studies [24]–[26], [29] have made considerable efforts to generate valid or diverse test cases through relation reasoning or constraint extraction. Then, IvySyn [8] performs type-aware and mutation fuzz testing on low-level kernel code in DL frameworks to detect vulnerabilities. TENSORSCOPE [9]

identifies corresponding APIs by leveraging model transformation rules and performs cross-framework testing. TitanFuzz [30] leverages zero-shot prompts with LLMs to generate seeds and perform test case mutation. FuzzGPT [31] uses historical bug data to prompt and fine-tune LLMs for generating unusual code. Orion [32] analyzes historical bugs to build fuzzing heuristic rules and generate corner-case test inputs for APIs. Recently, YanHui [33] proposes a domain knowledge-aware prompting method that uses LLMs to detect model optimization bugs. D^3 [74] uses distributed equivalence rules to test the distributed training and inference functionalities in DL frameworks. Unlike the above studies, MirrorFuzz reveals and analyzes the phenomenon of shared bugs between APIs across frameworks, leveraging this insight for early bug detection between frameworks.

VIII. FUTURE WORK

In future work, we plan to extend our approach in several directions to address current limitations.

Towards Adaptive Thresholds. In our work, we combine top- k and a similarity threshold to filter candidate API matches. The threshold is based on empirical observation, derived from evaluating candidate options and validating on API samples from multiple DL frameworks with varying functional complexities. While this approach works in many cases, it may not be optimal in all scenarios. In future work, we plan to explore the factors that influence the threshold H and conduct a formal analysis of their relationships to construct an adaptive threshold that adapts to different scenarios. In addition, we will leverage LLM-based agents to assist in filtering and refining candidate API matches, thereby improving the overall matching quality.

Discovering Novel Bug Classes. At present, our design relies on existing bug reports to detect similar bugs across DL frameworks. This reliance may limit the system's capacity to discover entirely novel bug classes (i.e., "unknown unknowns"). To improve MirrorFuzz's capability in finding new bugs, we adopted heuristic mutation operators and randomized scheduling to increase the diversity of generated test cases. Furthermore, newly detected bugs are automatically incorporated into the bug records, allowing the scope of shared bug patterns to gradually expand over time. In future work, we plan to explore adaptive mutation operators to further increase the diversity of test cases, thereby helping to uncover previously unknown bugs. We also plan to explore vulnerability-aware fuzzing strategies for specific bug types, such as use-after-free, integer overflow, and buffer overflow, to further discover novel bug classes in DL frameworks.

Improving Robustness and Adaptability with LLM Agents. The varying quality of historical bug reports poses a challenge for LLMs in the recognition of buggy APIs. This problem is particularly common in DL frameworks with less mature community ecosystems, such as OneFlow and Jittor, where bug reports are often ambiguous or incomplete. To mitigate this challenge, we employ targeted data preprocessing and carefully designed few-shot CoT prompts. In future work, we plan to construct approaches driven by LLM-based agents,

decomposing the workflow into sub-tasks such as bug report filtering, bug report normalization and enrichment, and final buggy API identification to enhance the analytical consistency and robustness of our approach.

In addition, some components in the current design, such as keyword-based filtering, regex extraction, and Levenshtein Distance matching, were designed based on practical experience and have been shown to be effective in our experiments. However, they may introduce brittleness and limit long-term extensibility. With the rapid evolution of LLM capabilities, we plan to migrate the MirrorFuzz pipeline to a multi-agent-based paradigm. In this paradigm, specialized agents replace these brittle components and handle key tasks such as buggy API recognition and similar API matching, to enhance the robustness and adaptability of MirrorFuzz.

IX. CONCLUSION

In this paper, we reveal that bugs found in the APIs of one framework may pose a threat to similar APIs within and across frameworks. Based on this insight, we propose MirrorFuzz, a novel approach that leverages shared bugs for early bug detection. MirrorFuzz employs LLM to identify buggy APIs from bug reports and synthesizes targeted test cases for similar APIs to induce similar bugs. We evaluate MirrorFuzz on four popular DL frameworks and discover 315 bugs, 262 of which are newly discovered, with 52 assigned CNVD IDs. Moreover, MirrorFuzz outperforms state-of-the-art fuzzers in bug detection, with code coverage improvements of 39.92% on TensorFlow and 98.20% on PyTorch, respectively.

REFERENCES

- [1] S. Shamshirband, M. Fathi, A. Dehzeni, A. T. Chronopoulos, and H. Alinejad-Rokny, "A review on deep learning approaches in healthcare systems: Taxonomies, challenges, and open issues," *Journal of Biomedical Informatics*, vol. 113, p. 103627, 2021.
- [2] M. Cascella, J. Montomoli, V. Bellini, and E. Bignami, "Evaluating the feasibility of chatgpt in healthcare: an analysis of multiple clinical and research scenarios," *Journal of medical systems*, vol. 47, no. 1, p. 33, 2023.
- [3] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–20.
- [4] R. Balse, V. Kumar, P. Prasad, and J. M. Warriem, "Evaluating the quality of llm-generated explanations for logical errors in cs1 student programs," in *Proceedings of the 16th Annual ACM India Compute Conference*, 2023, pp. 49–54.
- [5] Y. Li, S. Wang, H. Ding, and H. Chen, "Large language models in finance: A survey," in *Proceedings of the fourth ACM international conference on AI in finance*, 2023, pp. 374–382.
- [6] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of field robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [7] G. Go, C. Zhou, Q. Zhang, X. Zou, H. Shi, and Y. Jiang, "Towards more complete constraints for deep learning library testing via complementary set guided refinement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1338–1350.
- [8] N. Christou, D. Jin, V. Atlidakis, B. Ray, and V. P. Kemerlis, "[IvySyn]: Automated vulnerability discovery in deep learning frameworks," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2383–2400.
- [9] Z. Deng, G. Meng, K. Chen, T. Liu, L. Xiang, and C. Chen, "Differential testing of cross deep learning framework {APIs}: Revealing inconsistencies and vulnerabilities," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 7393–7410.

- [10] H. Guan, Y. Xiao, J. Li, Y. Liu, and G. Bai, "A comprehensive study of real-world bugs in machine learning model optimization," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 147–158.
- [11] F. Tambon, A. Nikanjam, L. An, F. Khomh, and G. Antoniol, "Silent bugs in deep learning frameworks: an empirical study of keras and tensorflow," *Empirical Software Engineering*, vol. 29, no. 1, p. 10, 2024.
- [12] R. Zhu, G. Chen, W. Shen, X. Xie, and R. Chang, "My model is malware to you: Transforming ai models into malware by abusing tensorflow apis," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2024, pp. 12–12.
- [13] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [14] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "{TensorFlow}: a system for {Large-Scale} machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.
- [15] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 1110–1121.
- [16] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 510–520.
- [17] J. Chen, Y. Liang, Q. Shen, J. Jiang, and S. Li, "Toward understanding deep learning framework bugs," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–31, 2023.
- [18] X. Zhang, W. Jiang, C. Shen, Q. Li, Q. Wang, C. Lin, and X. Guan, "A survey of deep learning library testing methods," *arXiv preprint arXiv:2404.17871*, 2024.
- [19] H. V. Pham, T. Lutellier, W. Qi, and L. Tan, "Cradle: cross-backend validation to detect and localize bugs in deep learning libraries," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1027–1038.
- [20] Z. Wang, M. Yan, J. Chen, S. Liu, and D. Zhang, "Deep learning library testing via effective model generation," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 788–799.
- [21] Q. Guo, X. Xie, Y. Li, X. Zhang, Y. Liu, X. Li, and C. Shen, "Audee: Automated testing for deep learning frameworks," in *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*, 2020, pp. 486–498.
- [22] J. Gu, X. Luo, Y. Zhou, and X. Wang, "Muffin: Testing deep learning libraries via neural architecture fuzzing," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1418–1430.
- [23] J. Liu, Y. Huang, Z. Wang, L. Ma, C. Fang, M. Gu, X. Zhang, and Z. Chen, "Generation-based differential fuzzing for deep learning libraries," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–28, 2023.
- [24] A. Wei, Y. Deng, C. Yang, and L. Zhang, "Free lunch for testing: Fuzzing deep-learning libraries from open source," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 995–1007.
- [25] Y. Deng, C. Yang, A. Wei, and L. Zhang, "Fuzzing deep-learning libraries via automated relational api inference," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 44–56.
- [26] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, and M. W. Godfrey, "Docter: Documentation-guided fuzzing for testing deep learning api functions," in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, 2022, pp. 176–188.
- [27] J. Wang, T. Lutellier, S. Qian, H. V. Pham, and L. Tan, "Eagle: creating equivalent graphs to test deep learning libraries," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 798–810.
- [28] C. Yang, Y. Deng, J. Yao, Y. Tu, H. Li, and L. Zhang, "Fuzzing automatic differentiation in deep-learning libraries," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1174–1186.
- [29] J. Shi, Y. Xiao, Y. Li, Y. Li, D. Yu, C. Yu, H. Su, Y. Chen, and W. Huo, "Acetest: Automated constraint extraction for testing deep learning operators," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 690–702.
- [30] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.
- [31] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [32] N. Shiri Harzevili, M. M. Mohajer, M. Wei, H. V. Pham, and S. Wang, "History-driven fuzzing for deep learning libraries," *ACM Transactions on Software Engineering and Methodology*.
- [33] H. Guan, G. Bai, and Y. Liu, "Large language models can connect the dots: Exploring model optimization bugs with domain knowledge-aware prompts," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1579–1591.
- [34] Y. Oliinyk, M. Scott, R. Tsang, C. Fang, H. Homayoun *et al.*, "Fuzzing busybox: Leveraging llm and crash reuse for embedded bug unearthing," *arXiv preprint arXiv:2403.03897*, 2024.
- [35] Z. Li, J. Wu, X. Ling, T. Luo, Z. Rui, and Y. Wu, "The seeds of the future sprout from history: Fuzzing for unveiling vulnerabilities in prospective deep-learning libraries," *arXiv preprint arXiv:2412.01317*, 2024.
- [36] N. H. Pham, T. T. Nguyen, H. A. Nguyen, and T. N. Nguyen, "Detection of recurring software vulnerabilities," in *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*, 2010, pp. 447–456.
- [37] W. Kang, B. Son, and K. Heo, "Tracer: Signature-based static analysis for detecting recurring vulnerabilities," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1695–1708.
- [38] "Oneflow," <https://github.com/Oneflow-Inc/oneflow>.
- [39] S.-M. Hu, D. Liang, G.-Y. Yang, G.-W. Yang, and W.-Y. Zhou, "Jittor: a novel deep learning framework with meta-operators and unified graph execution," *Science China Information Sciences*, vol. 63, no. 222103, pp. 1–21, 2020.
- [40] P. Ji, Y. Feng, D. Wu, L. Yan, P. Chen, J. Liu, and Z. Zhao, "Moco: Fuzzing deep learning libraries via assembling code," *IEEE Transactions on Software Engineering*, 2024.
- [41] "The github repository of mirrorfuzz," <https://github.com/MirrorFuzz/MirrorFuzz>.
- [42] D. Lenton, F. Pardo, F. Falck, S. James, and R. Clark, "Ivy: Templated deep learning for inter-framework portability," *arXiv preprint arXiv:2102.02886*, 2021.
- [43] "Candle," <https://github.com/huggingface/candle>.
- [44] J. Heek, A. Levskaya, A. Oliver, M. Ritter, B. Rondepierre, A. Steiner, and M. van Zee, "Flax: A neural network library and ecosystem for JAX," 2024. [Online]. Available: <http://github.com/google/flax>
- [45] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [46] C. Team, H. Zhao, J. Hui, J. Howland, N. Nguyen, S. Zuo, A. Hu, C. A. Choquette-Choo, J. Shen, J. Kelley *et al.*, "Codegemma: Open code models based on gemma," *arXiv preprint arXiv:2406.11409*, 2024.
- [47] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Dang *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.
- [48] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [49] ONNX, "Open neural network exchange," <https://onnx.ai/>.
- [50] CVEDetails, "Pytorch vulnerability statistics," <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=pytorch>.
- [51] GithubIssues, "Github issue of pytorch," <https://github.com/pytorch/pytorch/issues>.
- [52] Github, "Github rest api for issue," <https://docs.github.com/en/rest/issues>.
- [53] Google, "Google colab," <https://colab.research.google.com/>.
- [54] Tree-sitter Contributors, "Tree-sitter: An incremental parsing system for programming tools," <https://github.com/tree-sitter/tree-sitter>.
- [55] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

- [56] Wikipedia, “Levenshtein distance,” https://en.wikipedia.org/wiki/Levenshtein_distance.
- [57] —, “term frequency-inverse document frequency,” <https://en.wikipedia.org/wiki/Tf-idf>.
- [58] —, “Cosine similarity,” https://en.wikipedia.org/wiki/Cosine_similarity.
- [59] S. Qaiser and R. Ali, “Text mining: use of tf-idf to examine the relevance of words to documents,” *International Journal of Computer Applications*, vol. 181, no. 1, pp. 25–29, 2018.
- [60] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 2318–2335.
- [61] L. Cai, H. Wang, Q. Huang, X. Xia, Z. Xing, and D. Lo, “Biker: a tool for bi-information source based api method recommendation,” in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 1075–1079.
- [62] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Transactions on Information Systems*, 2023.
- [63] “bs4,” <https://pypi.org/project/beautifulsoup4/>.
- [64] Github, “requests,” <https://github.com/psf/requests>.
- [65] “Hugging face,” <https://huggingface.co>.
- [66] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [67] “coverage.py,” <https://github.com/nedbat/coveragepy>.
- [68] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, “{AddressSanitizer}: A fast address sanity checker,” in *2012 USENIX annual technical conference (USENIX ATC 12)*, 2012, pp. 309–318.
- [69] J. Cao, B. Chen, C. Sun, L. Hu, S. Wu, and X. Peng, “Understanding performance problems in deep learning systems,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 357–369.
- [70] J. Liu, J. Lin, F. Ruffy, C. Tan, J. Li, A. Panda, and L. Zhang, “Nnsmith: Generating diverse and valid test cases for deep learning compilers,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 530–543.
- [71] J. Liu, J. Peng, Y. Wang, and L. Zhang, “Neuri: Diversifying dnn generation via inductive rule inference,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 657–669.
- [72] X. Zhang, J. Liu, N. Sun, C. Fang, J. Liu, J. Wang, D. Chai, and Z. Chen, “Duo: Differential fuzzing for deep learning operators,” *IEEE Transactions on Reliability*, vol. 70, no. 4, pp. 1671–1685, 2021.
- [73] “Keras,” <https://keras.io/>, 2015.
- [74] J. Wang, H. V. Pham, Q. Li, L. Tan, Y. Guo, A. Aziz, and E. Meijer, “D 3: Differential testing of distributed deep learning with model generation,” *IEEE Transactions on Software Engineering*, 2024.